

anndataoom: out-of-memory AnnData for million-cell single-cell pipelines

Zehua Zeng^{1,2} and anndata-oom contributors¹

¹OmicVerse Project

²starlitnightly@163.com | <https://github.com/omicverse/anndata-oom>

Abstract

Single-cell RNA-sequencing atlases now routinely exceed one million cells, but the **AnnData** object¹ that scanpy operates on² holds the full expression matrix in memory, and the canonical preprocessing pipeline (quality control → normalize → log1p → scale → PCA) materialises a dense, z -scored matrix that exhausts commodity RAM at this scale. We present **anndataoom**, a drop-in out-of-memory storage backend for **AnnData** that keeps the matrix on disk through a Rust-native h5ad reader and composes normalisation, $\log(1+x)$ and scaling as lazy, chunk-wise transforms, so that peak resident memory is bounded by the chunk size rather than by the number of cells. Integration with the **omicverse** pipeline³ is transparent: the same `ov.pp.{qc, preprocess, scale, pca}` calls detect the backend through a single dispatch and route to a chunked implementation with identical numerical semantics. Benchmarking four storage/scaling configurations on seven real Tabula Sapiens slices⁴ from cellxgene⁵ spanning 5,000 to 1,053,033 cells, the out-of-memory backend holds peak memory nearly flat across dataset sizes while the in-memory backends grow linearly and are killed by a 256 GB per-process cap beyond 592,317 cells; it is the only configuration that completes 1,053,033 cells, finishing in 44.8 min at 5.0 GB. Principal components match the in-memory pipeline up to sign. The backend composes with GPU-accelerated execution and a portable Rust build, providing a practical route to whole-atlas preprocessing on a single commodity node.

Introduction

A typical scRNA-seq dataset has grown from $\sim 10^3$ cells to 10^6 – 10^7 in contemporary atlases (Tabula Sapiens⁴; CZ CELLxGENE⁵), yet the shape of the standard analysis is unchanged: a cells-by-genes matrix is quality-filtered, normalised to a target library size, $\log(1+x)$ -transformed, z -scored across cells, and projected to 50 principal components. The pre-PCA scaled matrix is *dense*: at 10^6 cells $\times$ 2,000 highly variable genes (HVGs) in `float32` it already occupies 8 GB, and standard implementations allocate several transient copies during scaling, pushing peak resident set size (RSS) well beyond what most workstations provide. The full in-memory **AnnData** must additionally be held in RAM.

Existing approaches either change the user-facing API, demand a different storage backend (Zarr, TileDB-SOMA), or partially materialise the matrix during preprocessing. We take a different route: preserve the **AnnData** contract, swap only the storage backend, and *compose* the expensive per-cell transforms as lazy operations over chunked reads, so the user-facing pipeline is unchanged.

Contributions. (i) A drop-in out-of-memory **AnnData** backend that streams sparse rows from h5ad through a Rust I/O layer and exposes chunked aggregations without materialising the matrix; (ii) a lazy transform chain in which normalisation, $\log(1+x)$ and scaling compose as nested wrappers applied per chunk during iteration; (iii) transparent **omicverse** integration through a single backend-detection dispatch; (iv) a parallel benchmark of four configurations on seven real Tabula Sapiens slices showing nearly flat peak RSS for the out-of-memory backend versus linear growth and eventual out-of-memory failure for the in-memory backends; and (v) an assessment of GPU-mixed execution, function-level compatibility, and cross-platform support.

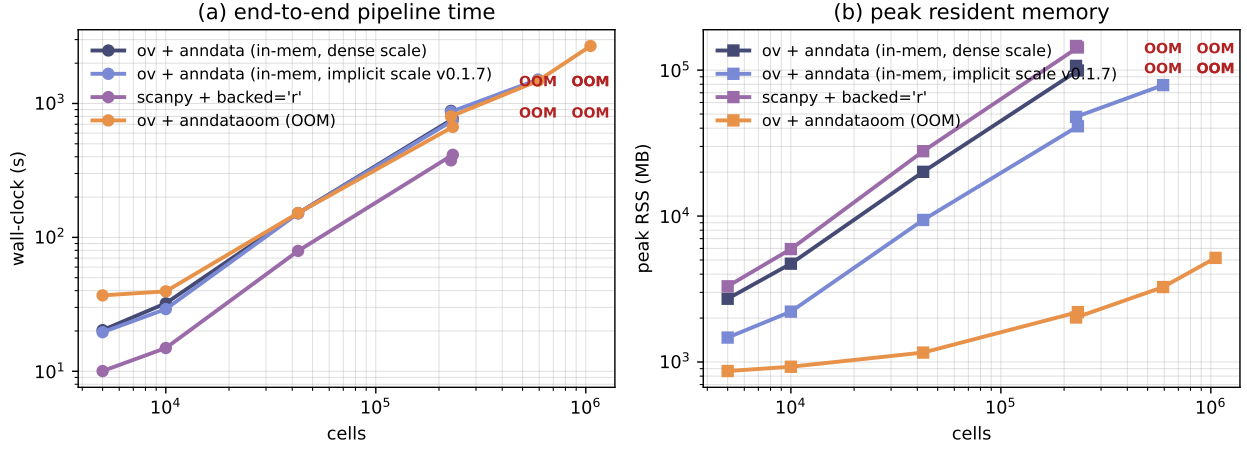


Fig. 1. Fig. 1 | Memory and runtime scale differently across storage backends. End-to-end pipeline wall-clock (left) and peak RSS (right) versus dataset size, log-log, for the four configurations. The out-of-memory backend’s nearly flat RSS reflects chunk-bounded allocation; the in-memory backends grow linearly via the dense scaling step and are killed at the 256 GB per-process cap for $\geq 592,317$ cells (annotated “OOM”).

Results

An out-of-memory AnnData backend with lazy transforms

`anndataoom` wraps a Rust `h5ad` reader⁶ in a `BackedArray` proxy that exposes the standard `shape`, `dtype`, `T` and indexing interface plus a `chunked(chunk_size)` iterator yielding `(start, end, chunk)` triples; for sparse inputs the chunks remain `scipy.sparse` matrices and the proxy never densifies on read. The three expensive per-cell transforms decompose into element- or row-wise operations applied independently per chunk. `TransformedBackedArray` applies per-cell normalisation (left-multiplication by a diagonal of inverse size factors, preserving sparsity) and $\log(1+x)$ (applied to the stored values only, since $\log(1+0) = 0$), at a memory cost of $O(n_{\text{obs}})$ for the factor vector and nothing else. `ScaledBackedArray` adds z -scoring, storing only the per-gene mean and standard-deviation vectors (~ 16 KB for 2,000 genes); the dense scaled block is reconstructed one chunk at a time at the consumer (typically PCA). `omicverse`’s `qc`, `preprocess`, `scale` and `pca` functions detect the backend with a single `is_oom(adata)` check and dispatch to the chunked path, so user code does not branch on backend type.

Memory stays flat across dataset size

We benchmarked the identical pipeline (`qc`, mode `seurat`; `preprocess`, `shiftlog|pearson`, 2,000 HVGs; `scale`, `max_value=10`; `pca`, 50 components) under four configurations that differ only in storage/scaling (**Methods**): the `omicverse` pipeline on an in-memory `AnnData` with dense scaling (`ov-anndata`); the same with an implicit-centered scaled wrapper (`ov-anndata-implicit`); the `scanpy` pipeline opened `backed='r'` (`scanpy-backed`); and the `omicverse` pipeline on `anndataoom` (`ov-oom`). Inputs are seven *Tabula Sapiens* slices on a shared 60,606-gene panel spanning 5,000 to 1,053,033 cells (Table 1).

Three regimes emerge (Fig. 1; Table 2). *Below the in-memory frontier* ($\leq 228,032$ cells) all four complete; at TS-Epi (228,032 cells) peak RSS is 104, 47, 143 and 2.0 GB for `ov-anndata`, `ov-anndata-implicit`, `scanpy-backed` and `ov-oom` respectively — the out-of-memory backend uses $53.0\times$ less RAM than the in-memory default. *Past 592,317 cells* both `anndata`-only paths are killed by the 256 GB per-process cap; the implicit-centered wrapper extends the in-memory frontier to 592,317 cells (25.2 min, 77 GB). *At 1,053,033 cells* only `ov-oom` completes (44.8 min, 5.0 GB). The flat RSS curve on the out-of-memory side reflects the bounded per-chunk allocations of the lazy transform chain.

Table 1. Benchmark datasets. Seven real Tabula Sapiens slices⁴ from cellxgene⁵ on the shared 60,606-gene panel, with `layers["decontXcounts"]` (DecontX-decontaminated counts) promoted to `.X`.

Dataset	Cells	Genes	On-disk (MB)	Source
TS-5k	5,000	60,606	130.1	Tabula Sapiens vasculature (sub)
TS-10k	10,000	60,606	222.5	Tabula Sapiens vasculature (sub)
TS-Vasc	42,650	60,606	2081.8	Tabula Sapiens vasculature
TS-Stromal	232,684	60,606	9594.1	Tabula Sapiens stromal compartment
TS-Epi	228,032	60,606	11066.5	Tabula Sapiens epithelial compartment
TS-Immune	592,317	60,606	18858.0	Tabula Sapiens immune compartment
TS-1M	1,053,033	60,606	15439.3	Tabula Sapiens stromal+epi+immune (concat)

Table 2. End-to-end wall-clock and peak RSS for each (configuration, dataset) cell. “`ov-anndata*`” denotes the implicit-centered scaling path. “OOM” marks runs killed by the 256 GB per-process cap.

	ov-anndata		ov-anndata*		scanpy-backed		ov-oom	
	t (s)	RSS (MB)	t (s)	RSS (MB)	t (s)	RSS (MB)	t (s)	RSS (MB)
TS-5k	20.3	2711.5	19.6	1470.1	10.0	3300.6	36.9	866.7
TS-10k	32.2	4716.8	29.1	2213.7	14.9	5940.4	39.5	927.8
TS-Vasc	151.7	20095.3	151.2	9404.1	79.2	27821.5	151.9	1160.2
TS-Stromal	766.9	99371.0	745.9	41229.6	413.9	142962.9	668.5	2196.2
TS-Epi	880.1	106938.9	866.9	47831.9	375.8	146379.4	800.7	2017.6
TS-Immune	OOM	–	1509.0	78949.4	OOM	–	1476.6	3261.8
TS-1M	OOM	–	OOM	–	OOM	–	2688.7	5168.7

Per-stage cost and numerical equivalence

Decomposing by stage (Fig. 2), the out-of-memory backend’s wall-clock is dominated by PCA, while the in-memory backends are dominated by the dense scaling step and PCA. Normalisation and $\log(1+x)$ are effectively free in the out-of-memory path because both are absorbed into the lazy wrapper and execute only during the next consumer’s chunked read. The two backends produce identical principal components up to a per-component sign flip inherent to the SVD: per-component cosine similarity ≥ 0.9999 on every dataset where both completed.

Composes with GPU-mixed execution at no penalty

`omicverse` offers a `cpu-gpu-mixed` execution mode that offloads PCA, neighbour-graph construction and embedding to a GPU through PyTorch. Because `anndataoom` is a *storage* backend and mixed mode is a *compute* mode, the two are orthogonal. Running the out-of-memory pipeline under mixed mode on an NVIDIA H100, the quality-control-to-PCA pipeline is *mode-invariant*: at TS-1M (1,053,033 cells) wall-clock moves only $2689 \rightarrow 2811$ s and peak RSS is unchanged (5.0 vs 4.8 GB), with *no* GPU memory allocated during these stages (Table 3). This follows structurally: the chunked operators are CPU-resident, and the out-of-memory PCA dispatches to a chunked solver rather than the GPU path, so the accelerator is never engaged for the memory-bound work that the backend exists to make tractable — and peak RSS stays flat regardless of mode. GPU offload does engage for the downstream graph and embedding steps, which operate on the compact ($n_{\text{obs}} \times 50$) PCA embedding and never touch the matrix. On an in-memory backend GPU offload of PCA gave no consistent speedup at these scales, as the CPU covariance-eigendecomposition solver already completes the 2,000-HVG decomposition in well under ten seconds. The out-of-memory backend therefore composes with mixed mode while preserving its flat-memory guarantee.

Function coverage and cross-platform support

Beyond the four pipeline stages, 14 of 24 probed `omicverse.pp` functions run on the out-of-memory backend (Table 4), covering the full canonical workflow (`qc`, `preprocess`, `normalize_total`, `log1p`,

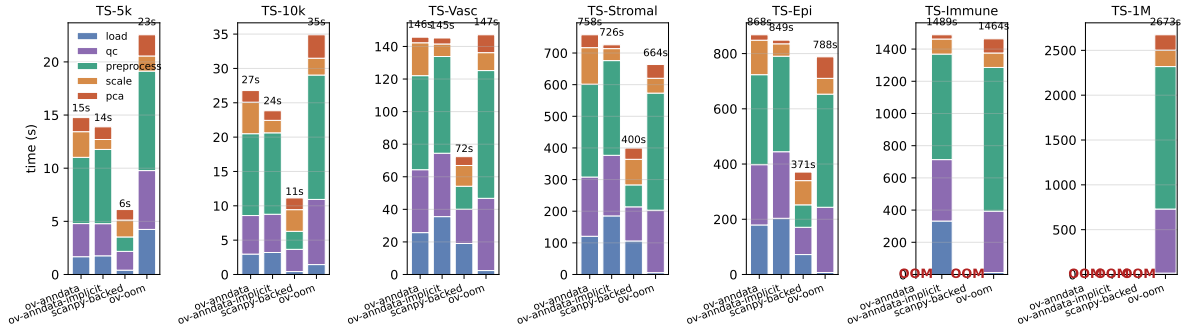


Fig. 2. Fig. 2 | Per-stage wall-clock breakdown across all benchmark points. The out-of-memory normalize stage is effectively free; its cost is amortised into the next chunked read.

Table 3. CPU vs CPU-GPU-mixed on the out-of-memory and in-memory backends across scale. “GPU (MB)” is peak PyTorch device memory during the four-stage pipeline; it is ≈ 0 for the out-of-memory backend because its PCA path is CPU-resident.

Backend	Dataset	cpu		cpu-gpu-mixed			speedup t_{cpu}/t_{mix}
		t (s)	RSS (MB)	t (s)	RSS (MB)	GPU (MB)	
OOM (anndataoom)	TS-5k	36.9	866.7	24.4	922.9	0.0	1.51
OOM (anndataoom)	TS-10k	39.5	927.8	38.0	936.9	0.0	1.04
OOM (anndataoom)	TS-Vasc	151.9	1160.2	144.9	1125.8	0.0	1.05
OOM (anndataoom)	TS-Stromal	668.5	2196.2	682.3	2028.0	0.0	0.98
OOM (anndataoom)	TS-Epi	800.7	2017.6	821.5	2028.7	0.0	0.97
OOM (anndataoom)	TS-Immune	1476.6	3261.8	1371.0	3272.7	0.0	1.08
OOM (anndataoom)	TS-1M	2688.7	5168.7	2810.7	4940.9	0.0	0.96
in-memory dense	TS-5k	20.3	2711.5	21.1	3144.1	324.4	0.96
in-memory dense	TS-10k	32.2	4716.8	32.3	5124.2	400.7	1.00
in-memory dense	TS-Vasc	151.7	20095.3	150.7	20365.2	898.9	1.01
in-memory dense	TS-Stromal	766.9	99371.0	849.1	99104.4	3798.6	0.90
in-memory dense	TS-Epi	880.1	106938.9	923.7	106605.5	3728.1	0.95

identify_robust_genes, scale, pca) and the graph, clustering and embedding steps (neighbors, leiden, louvain, umap, tsne, mde, sude). The remaining functions are not yet wired through the backend-detection dispatch and raise a clear exception at the call site rather than silently miscomputing. The Rust extension, which links HDF5, builds and passes its test suite on Linux, macOS (Apple-silicon and Intel) and Windows across Python 3.10 and 3.12 in continuous integration.

Discussion

For real-data sizes from 5,000 cells upward the out-of-memory backend matches or beats the in-memory pipeline’s wall-clock while using a small fraction of the memory, because it avoids loading and densifying the full gene panel; past the in-memory frontier it continues without intervention, with peak RSS bounded by chunk size rather than by cell count. The cost is that lazy, on-disk evaluation re-reads from disk at each stage, so for workflows that fit comfortably in RAM and re-use a loaded `AnnData` many times within a session, the amortised cost of the in-memory backend can be lower. The backend is a storage-layer change only — it does not alter the analysis or its numerics — so it composes with GPU-accelerated compute and with the broader `omicverse` ecosystem. The natural extensions are broader backend-adapted coverage of auxiliary functions, and multi-omic layers (ATAC peaks, protein counts) following the same lazy-wrap pattern.

Table 4. `omicverse.pp` compatibility on the out-of-memory backend. ✓ = runs to completion; ✗ = raises (not yet backend-adapted); superscript ^G marks a call that offloads to the GPU under `cpu-gpu-mixed`.

ov.pp function	cpu	mixed	GPU-offload	note
qc	✓	✓	—	
preprocess	✓	✓	—	
scale	✓	✓	—	
pca	✓	✓	—	
neighbors	✓	✓ ^G	yes	
leiden	✓	✓	—	
louvain	✓	✓	—	
umap	✓	✓ ^G	yes	
tsne	✓	✓	—	
mde	✓ ^G	✓ ^G	yes	torch (GPU-capable)
sude	✓	✗	—	fails in mixed (NaN)
normalize_total	✓	✓	—	
log1p	✓	✓	—	
identify_robust_genes	✓	✓	—	
highly_variable_features	✗	✗	—	pegasus HVG densifies
highly_variable_genes	✗	✗	—	standalone; use <code>preprocess</code>
normalize_pearson_residuals	✗	✗	—	not OOM-adapted
regress	✗	✗	—	scanpy <code>regress_out</code> densifies
scrublet	✗	✗	—	backed var lookup
score_genes_cell_cycle	✗	✗	—	backed var lookup
filter_cells	✗	✗	—	not OOM-adapted
filter_genes	✗	✗	—	not OOM-adapted
anndata_to_GPU	✗	✗	—	needs rapids
anndata_to_CPU	✗	✗	—	needs rapids

Methods

Storage and transform layer. The h5ad reader is the `anndata-rs` Rust library, exposed to Python as a `BackedArray` proxy. Per-cell normalisation is implemented as left multiplication by a sparse diagonal of inverse size factors; $\log(1+x)$ is applied to the sparse values only. Per-gene mean and variance for scaling are computed in a single chunked pass using a hybrid estimator: per-chunk statistics use the sparse-native $E[X^2] - E[X]^2$ identity, and cross-chunk accumulation uses Welford’s method⁷ on the per-chunk mean and sum-of-squared-deviations, keeping the numerical error at the `float32`→`float64` accumulation floor ($\sim 7 \times 10^{-8}$ relative).

Chunked PCA. Dimensionality reduction uses a randomised SVD⁸ over the lazy scaled matrix. When the post-HVG matrix fits a configurable memory budget it is built once into RAM and factorised in a single pass; otherwise an implicit-centered formulation rewrites the matrix–vector products through the identity $(N - \mu)/\sigma \cdot W = N \cdot \text{diag}(1/\sigma) \cdot W - (\mu/\sigma) \cdot W$, where N is the sparse normalise+ $\log(1+x)$ view, so no dense per-chunk block is allocated. When the matrix is materialised, HVG columns are sliced from the sparse parent *before* densifying — since normalisation is per-row and $\log(1+x)$ is element-wise, the column slice commutes through both — so only a $(\text{chunk} \times n_{\text{HVG}})$ block is ever dense; this is numerically identical to densifying the full panel (maximum absolute difference $\sim 10^{-7}$).

Benchmark configurations. `ov-anndata`: input loaded with `anndata.read_h5ad`; `ov.pp.scale` densifies the full panel. `ov-anndata-implicit`: as above but `ov.pp.scale` stores the scaled matrix as a sparse-backed implicit-centered wrapper. `scanpy-backed`: input opened `backed='r'`, then the standard scanpy pipeline; because the first quality-control operation has no handler for the backed dataset type, the workflow materialises to memory before QC, so the backed advantage exists only at load. `ov-oom`: input opened with `anndataoom.read`; operations dispatch to the chunked path via `is_oom`.

Datasets. Seven Tabula Sapiens slices⁴ from cellxgene⁵ on the shared 60,606-gene panel: TS-Vasculature (42,650 cells) and two random subsamples (5,000 / 10,000, seed 0); the stromal (232,684), epithelial (228,032) and immune (592,317) compartments; and a one-million-cell concatenation (1,053,033 cells). Each input uses `layers["decontXcounts"]` as `.X`; the harness promotes the raw-counts layer when `.X` appears log-normalised, to avoid double-normalisation.

Measurement. Per-stage wall-clock and RSS are captured around each pipeline stage (peak RSS reported as the maximum post-stage RSS). The main sweep runs on a single CPU node under a 256 GB per-process cap; the GPU-mixed comparison runs on an NVIDIA H100 node, recording per-stage peak PyTorch device memory in addition to RSS. Numerical equivalence is quantified by per-component absolute cosine similarity between backends.

Code and data availability. The benchmark harness, figure/table generators and dataset cards are at the project repository⁶. Subsamples use a fixed seed; the million-cell input is the row-concatenation of the three compartment slices.

References

- [1] Isaac Virshup, Sergei Rybakov, Fabian J Theis, Philipp Angerer, and F Alexander Wolf. anndata: Access and store annotated data matrices. *Journal of Open Source Software*, 9(101):4371, 2024.
- [2] F Alexander Wolf, Philipp Angerer, and Fabian J Theis. Scanpy: large-scale single-cell gene expression data analysis. *Genome Biology*, 19(1):15, 2018.
- [3] Zehua Zeng et al. Omicverse: a framework for bridging and deepening insights across bulk and single-cell sequencing. *Nature Communications*, 15:5983, 2024.
- [4] Tabula Sapiens Consortium et al. The tabula sapiens: A multiple-organ, single-cell transcriptomic atlas of humans. *Science*, 376(6594):eabl4896, 2022.
- [5] Colin Megill et al. cellxgene: a performant, scalable exploration platform for high-dimensional sparse matrices. In *bioRxiv*, pages 2021–04, 2021.
- [6] Zehua Zeng and Anthropic. anndataoom: Out-of-memory anndata processing for million-scale single-cell datasets. <https://github.com/omicverse/anndata-oom>, 2026.
- [7] B P Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3):419–420, 1962.
- [8] Nathan Halko, Per-Gunnar Martinsson, and Joel A Tropp. Finding structure with randomness: probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.